1. **List at least 10 features / functional requirements of your project**

Prepare a scenario/case study for your own project idea and provide feature list relevant to the proposed scenario.

Scenario / Case Study

Project Name: Bank Management System (EMS – EMU8086)

Objective:

To create a simple bank management system in assembly language (EMU8086) that allows a single user to create a bank account, deposit and withdraw money, view account details, reset, and modify account information. This project simulates a basic banking environment for learning purposes and demonstrates file handling, input/output, and account management in low-level programming.

Scenario:

Mr. Ali wants to open a bank account in a local bank. He wants to:

1. *Create a new account by providing his personal details.*
2. *Deposit money into his account in predefined amounts.*
3. *Withdraw money from his account while ensuring sufficient balance.*
4. *View his account details securely using a PIN.*
5. *Modify his account details if needed.*
6. *Reset his account in case he wants to start fresh.*

The system should ensure that only authorized access is allowed via a PIN, and all transactions are recorded in a file for persistence.

Feature List

Based on the code, the system provides the following features*:*

- *Create New Account*

User inputs account name, PIN, phone number, city, state, and pincode.

Account information is saved in a file automatically.

- *Retrieve Account Details*

User enters PIN to view account details.

Displays account name, phone number, city, state, and total balance.

Handles cases where no account is created or balance is zero.

- *Deposit Money*

Allows the user to deposit fixed amounts (Rs. 1000, 2000, 5000, 10000).

Updates total account balance.

- *Withdraw Money*

Allows the user to withdraw fixed amounts (Rs. 1000, 2000, 5000, 10000).

Checks for sufficient balance before withdrawing.

Displays success or error messages.

- *Reset Account*

Clears account information (name, PIN, balance).

Prepares the system for a fresh account creation.

- *Update / Modify Account*

Allows the user to modify the account name and PIN.

Changes are saved for future use.

- *PIN-based Security*

Users must enter the correct PIN to access account details, deposit, withdraw, reset, or modify account.

- *File Handling*

Account information is saved to a file for persistence.

Handles file open, write, seek, and close operations with error messages.

- *User-friendly Interface*

Displays menus with options (create account, deposit, withdraw, etc.).

Uses prompts, confirmation messages, and "press enter to continue" for smooth navigation.

- *Error Handling*

Handles errors like invalid PIN, insufficient balance, file access errors, and uninitialized accounts.

2. **Provide a feasibility study**

Prepare a table mapping each requirement to available Tools/resources/APIs/packages/libraries that would help you implement.

Requirement / Feature

Description

Tools / Resources / Libraries

1. *Create New Account*

User inputs account details (name, PIN, phone, city, state, pincode) and saves them.

EMU8086 input interrupts (int 21h), macros for input, file write using DOS interrupts (ah=40h)

2. *Retrieve Account Details*

Display account info and balance after verifying PIN.

DOS input/output (int 21h), loop and comparison logic, macros for PIN input masking

3. *Deposit Money*

Add selected predefined amounts to total balance.

Simple arithmetic (add instruction), input using int 21h, option selection via cmp/jmp

4. *Withdraw Money*

Subtract selected amount from balance; check sufficient funds.

Arithmetic (sub), comparison (cmp), conditional jumps (jl), input handling

5. *Modify Account*

Change account name or PIN.

Input macros (int 21h), string manipulation, updating stored values

6. *Reset Account*

Clear all account details and balance.

Direct memory overwrite (mov [si],' '), reset variables (totalAmount, accountPINCount)

7. *PIN Security*

Authenticate user before viewing/modifying account.

Loop to check each character of PIN, input masking (*), comparison (cmp)

8. *File Handling*

Save account info to file for persistence.

DOS file interrupts (int 21h): open (ah=3Dh), write (ah=40h), seek (ah=42h), close (ah=3Eh)

9. *User Interface / Menu*

Display menus and prompts to guide the user.

macro printString, DOS int 21h, loops for menu navigation, clear screen with blank lines

10. *Error Handling*

Handle invalid PIN, insufficient funds, or file errors.

Conditional jumps (jc, jne), messages stored in .data segment and displayed via int 21h

3. **Technical developer guide**

Choose any three features from your feature list and provide a detailed, step-by-step technical guide on how to implement them.

Technical Developer Guide

Feature 1: Create New Account

Purpose: Collect user information (name, PIN, phone, city, state, pincode) and save it to a file.

Step-by-step Implementation:

Display the Create Account Menu

Use printString createmsg to show "CREATE NEW ACCOUNT".

Use blank2 for spacing.

Get Account Details from User

Name ⟶ account_name accountName macro

PIN ⟶ account_pin accountPIN macro, track digit count with accountPINcount.

Phone ⟶ account_phone accountPhone macro

City ⟶ account_city accountCity macro

State ⟶ account_state accountState macro

Pincode ⟶ account_pincode accountPinCode macro

Store Data in File

Call the insert procedure.

Use DOS interrupts (int 21h) to open (ah=3Dh), seek (ah=42h), write (ah=40h), and close (ah=3Eh) the file.

Confirmation

Print "Press Enter to Confirm" and "Account Created".

Wait for user to press Enter (etc procedure).

Feature 2: Withdraw Money

Purpose: Allow user to withdraw a pre-defined amount if balance is sufficient.

Step-by-step Implementation:

Check if Account Exists

Call checkAccountCreated to ensure an account is created.

Ask for PIN verification (getPinInput macro).

Display Withdrawal Options

Print withdrawal menu (moneymsg1 to moneymsg4) for Rs.1000, Rs.2000, Rs.5000, Rs.10000.

Get User Selection

Call inputAmountCode to read input (int 21h) into inputAmountOption.

Check Balance and Subtract Amount

Compare balance (totalAmount) with selected withdrawal amount using cmp.

If enough → subtract and display success message (moneymsg7).

If not enough → display "Withdraw Limit Exceeded" (moneymsg6).

Return to Main Menu

After withdrawal, jump to mainLoop to show main menu again.

Feature 3: Deposit Money

Purpose: Allow user to deposit a pre-defined amount into their account.

Step-by-step Implementation:

Check Account and Verify PIN

Same as withdraw, call checkAccountCreated and getPinInput.

Display Deposit Options

Print deposit menu (moneymsg1 to moneymsg4).

Get User Selection

Call inputAmountCode to read user choice.

Add Selected Amount to Balance

Use add totalAmount, based on user selection (1000, 2000, 5000, 10000).

Confirmation

Display "Money Deposited Successfully" (moneymsg8).

Jump back to main menu (mainLoop).

4. **Implementation**

Using any ONE of the available tools, and implement the three selected features inside a small working project.

Create New Account

The system asks the user to enter their Name, PIN, Phone, City, State, and Pincode.

Each input is stored in separate variables in memory.

Once the account information is entered, it is saved to a text file using DOS file handling (INT 21h, function 40h for writing).

This ensures that account details persist even after the program is closed.

Deposit Money

After entering the correct account PIN, the user is shown preset deposit options (Rs.1000, 2000, 5000, 10000).

The system adds the selected amount to the totalAmount variable.

A success message confirms the deposit.

This feature uses simple arithmetic (ADD) and user input via keyboard interrupts.

Withdraw Money

After PIN verification, the user selects an amount to withdraw from preset options.

The system checks if the account has sufficient balance before subtracting the amount.

If the balance is enough, the withdrawal is completed and the totalAmount is updated.

If not, a warning message is displayed, preventing overdraft.

Tool Used: EMU8086 (8086 Microprocessor Emulator)

Used DOS interrupts (INT 21h) for input/output and file handling.

Variables in .data segment store account and transaction details.

Procedures and macros organize code for better readability and reusability.

Result:

All three features work together inside a small, functioning project where users can create an account, deposit, and withdraw money securely using EMU8086.

Code:

```
; ==========================

; Bank Management System

;       by Hashir

; ==========================

.model small

.stack 1000h

.data


    subject db 10,13,'MICROPORCESSOR AND INTERFACING $'


    course db 10,13,'CSE2006 $'


    heading db 10,13,  'BANK MANAGEMENT SYSTEM: USER INTERFACE $'


    exitmsg db 10,13, 'SYSTEM CLOSED! $'


    createmsg db 10,13, 'CREATE NEW ACCOUNT $'


    detailmsg db 10,13,'YOUR ACCOUNT DETAILS $'
```

```
withdrawmsg db 10,13, 'WITHDRAW $'

DEPOSITMSG db 10,13, 'DEPOSIT $'

resetmsg db 10,13,'CAUTION! YOU WILL RESET YOUR ACCOUNT $'

modmsg db 10, 13, 'CAUTION! YOU WILL PROCEED TO MODIFY YOUR ACCOUNT $'

mainmsg0 db 10,13,'0. Exit$'
mainmsg1 db 10,13,'1. Create New Account$'
mainmsg2 db 10,13,'2. Retrieve Account Details$'
mainmsg3 db 10,13,'3. Withdraw Money$'
mainmsg4 db 10,13,'4. Deposit Money$'
mainmsg5 db 10,13,'5. Reset Account$'
mainmsg6 db 10,13,'6. Update Account$'
mainmsg7 db 10,13,'Press Enter to return to main menu $'

detailmsg1 db 10,13, 'Account Name : $'
detailmsg2 db 10, 13, 'Your City : $'
detailmsg3 db 10, 13, 'No Accounts Currently Saved!!$'
```

```
detailmsg4 db 10, 13, 'Total Money Left : $'
detailmsg5 db 10, 13, 'Account Balance is Zero $'
detailmsg6 db 10, 13, 'Your State : $'
detailmsg7 db 10, 13, 'Your Phone No. : $'

moneymsg1 db 10,13,'1.  Rs. 1000$'
moneymsg2 db 10,13,'2.  Rs  2000$'
moneymsg3 db 10,13,'3.  Rs. 5000$'
moneymsg4 db 10,13,'4.  Rs. 10000$'
moneymsg5 db 10,13,'Enter Code >> $'
moneymsg6 db 10,13,'Withdraw Limit Exceeded. Process Terminated. $'
moneymsg7 db 10,13,'Money Withdrawn Successfully. $'
moneymsg8 db 10,13,'Money Deposited Successfully. $'

inputmsg db 'Choose an option >>  $'
inputCode db ?

accountName db 20 dup('$')
accountPIN db 20 dup('$')
accountPhone db 20 dup('$')
accountCity db 20 dup('$')
```

```
accountState db 20 dup('$')
accountPinCode db 20 dup('$')
accountPINcount dw 0        ;This keeps track how many digit a pin is
totalAmount dw 0
inputAmountOption db ?
resetDone db 10,13,'Account has been reset. $'

create1 db 10,13,'1. Enter Account Name : $'
create2 db 10,13,'2. Enter Account Pin :  $'
create5 db 10,13,'3. Enter Phone No. :  $'
create6 db 10,13,'4. Enter Your City :  $'
create7 db 10,13,'5. Enter Your State :  $'
create8 db 10,13,'6. Enter Address Pincode :  $'
create3 db 10,13,'Account Created. $'
create4 db 10,13,'Press Enter to Confirm. $'

pinMsg db 10,13,'Enter Pin >> $'

blank db 10,13,'>>  $'   ;for input blinker
blank2 db 10,13, '    $' ;For Newline
```

```
    modAccMsg1 db 10,13,'1. New Account Name ( Old: $'

    modAccMsg2 db ' ) : >> $'

    modPinMsg1 db 10,13,'2. New Account Pin ( Old: $'

    modPinMsg2 db ' ) : >>$'

    modSuccess db 10,13,'Account Details Updated. $'


    filename  db "..₩..₩Users₩S K Nayak₩Documents₩CSE2006
ProjectWadminWusers.txt", 0

    handle     dw ?

    usermsg    db 10, "hello world$"

    buffer     db 10 dup(?)

    msg_open  db 10, 13, "Error opening file!$"

    msg_seek  db 10, 13, "Error seeking file!$"

    msg_write db 10, 13, "Error writing file!$"

    msg_close db 10, 13, "Error closing file!$"

    new_line db 10,13


.code


;;;
```

```
;                    F U N C T I O N S                    ;
;                                                         ;
;;;
```

```
mov bx, handle

lea dx, new_line

mov cx, 2

mov ah, 40h

int 21h

lea dx, accountName

mov cx, 20

mov ah, 40h

int 21h ; write to file...

lea dx, accountPin

mov cx, 20

mov ah, 40h

int 21h ; write to file...

lea dx, accountPhone

mov cx, 20

mov ah, 40h

int 21h ; write to file...

lea dx, accountCity

mov cx, 20

mov ah, 40h

int 21h ; write to file...
```

```
    lea dx, accountState
    mov cx, 20
    mov ah, 40h
    int 21h ; write to file...
    lea dx, accountPinCode
    mov cx, 20
    mov ah, 40h
    int 21h ; write to file...
    jc err_write

    mov bx, handle
    mov ah, 3eh
    int 21h ; close file...
    jc err_close

err_open:
    lea dx, msg_open
    jmp error

err_seek:
    lea dx, msg_seek
```

```
   jmp error

err_write:
   lea dx, msg_write
   jmp error

err_close:
   lea dx, msg_close
   jmp error

error:
   ;mov ah, 09h
   ;int 21h
ret
endp insert


macro printString str
    lea dx, str
    mov ah, 09h
    ; mov dx, offset str
```

```
    int 21h
endm

displaySub proc near
    printString subject
    printString blank2
    printString course
    printString blank2

    ret
displaySub endp

displayHeading proc near
    printString heading
    printString blank2
    ret
displayHeading endp

displayinputMenu proc near
    printString mainmsg0
    printString mainmsg1
```

```
    printString mainmsg2
    printString mainmsg3
    printString mainmsg4
    printString mainmsg5
    printString mainmsg6
    printString mainmsg7
    printString blank2
    ret
displayinputMenu endp

inputMenu proc near
    printString inputmsg
    printString blank
    mov ah, 1
    int 21h
    mov inputCode, al
    ret
inputMenu endp

displayBye proc near
    printString exitmsg
```

```
    printString blank2
    ret
displayBye endp



waiting proc near
    mov cx, 0fh
    mov dx, 4240h
    mov ah, 86h
    int 15h
waiting endp

clearScreen proc near
    printString blank2
    printString blank2
    ret
clearScreen endp



;
##################################################################
```

```
################################################################
##

;
################################################################
################################################################
##

;  CREATE NEW ACCOUNT

;
################################################################
################################################################
##

;
################################################################
################################################################
##



macro account_name str

    mov si, offset str

    input:

        mov ah, 1

        int 21h

        cmp al, 13

        je create_pin

        mov [si], al
```

```
        inc si
        jmp input
    exitMac:
        ret
endm


macro account_pin str
    mov si, offset str
    input2:
        mov ah, 1
        int 21h
        cmp al, 13
        je create_phone
        inc accountPINcount
        mov [si], al
        inc si
        jmp input2
    exitMac2:
        ret
endm
```

```
macro account_phone str
    mov si, offset str
    input3:
        mov ah, 1
        int 21h
        cmp al, 13
        je create_city
        mov [si], al
        inc si
        jmp input3
    exitMac3:
        ret
endm


macro account_city str
    mov si, offset str
    input4:
        mov ah, 1
        int 21h
        cmp al, 13
        je create_state
```

```
        mov [si], al
        inc si
        jmp input4
    exitMac4:
        ret
endm


macro account_state str
    mov si, offset str
    input5:
        mov ah, 1
        int 21h
        cmp al, 13
        je create_pincode
        mov [si], al
        inc si
        jmp input5
    exitMac5:
        ret
endm
```

```
macro account_pincode str
    mov si, offset str
    input6:
        mov ah, 1
        int 21h
        cmp al, 13
        je account_created
        mov [si], al
        inc si
        jmp input6
    exitMac6:
        ret
endm

; Enter to continue
proc etc
   etcin:
      mov ah,1
      int 21h
      cmp al,13
      je mainloop
```

```
      jmp etcin
   ret
etc endp


create_account proc
    call clearScreen
    printString createmsg


    printString blank2
    printString create1
    printString blank
    account_name accountName



    create_pin:
        printString create2
        printString blank
        account_pin accountPIN


    create_phone:
        printString create5
```

```
    printString blank
    account_phone accountPhone

create_city:
    printString create6
    printString blank
    account_city accountCity

create_state:
    printString create7
    printString blank
    account_state accountState

create_pincode:
    printString create8
    printString blank
    account_pincode accountPinCode

account_created:
    printString create4
    printString create3
```

```
        call insert

        call etc


    ret

create_account endp


;
##################################################################
##################################################################
##

;
##################################################################
##################################################################
##

;  RETRIEVE ACCOUNT DETAILS

;
##################################################################
##################################################################
##

;
##################################################################
##################################################################
##


checkAccountCreated proc

    cmp accountPINCount, 0
```

```
    je accountNotCreated
    ret

    accountNotCreated:
        call clearScreen
        printString detailmsg3
        printString mainmsg7
        printString blank2
        call etc
checkAccountCreated endp

clearkeyboardbuffer       proc near
    clearin:
        mov ah, 1   ; peek
        int 16h
        jz  NoKey
        mov ah, 0   ; get
        int 16h
        jmp clearin:
    NoKey:
        ret
```

```
clearkeyboardbuffer        endp



getPinInput proc
    call clearScreen
    printString pinMsg
    printString blank

    mov si, offset accountPIN
    mov cx, accountPINCount

    getInput:
        mov ah, 7
        int 21h
        cmp al, [si]
        mov dl, '*'
        mov ah, 2
        int 21h
        jne mainLoop
        inc si
```

```
    loop getInput
    ret
getPinInput endp

printNumber proc
    mov cx, 0
    mov dx, 0

    label1:
        cmp ax, 0
        je print1
        mov bx, 10
        div bx
        push dx
        inc cx
        xor dx, dx
        jmp label1

    print1:
        cmp cx, 0
        je exitprint
```

```
        pop dx
        add dx, 48
        mov ah, 02h
        int 21h
        dec cx
        jmp print1

    exitprint:
        ret
printNumber endp

print_details proc
    call checkAccountCreated
    call getPinInput
    call clearScreen


    printString detailmsg
    printString blank2
    printString blank2
```

```
printString detailmsg1
printString accountName
printString blank2

printString detailmsg7
printString accountPhone
printString blank2

printString detailmsg2
printString accountCity
printString blank2

printString detailmsg6
printString accountState
printString blank2

printString detailmsg4
mov ax, totalAmount
cmp ax, 0
je noMoneyError
call printNumber
```

```
    printString mainmsg7
    printString blank2
    call etc

    noMoneyError:
        printString blank2
        printString detailmsg5
        printString mainmsg7
        printString blank2
        call etc
    ret
print_details endp

;
##################################################################
##################################################################
##

;
##################################################################
##################################################################
##

;  WITHDRAW MONEY

;
##################################################################
```

```
##################################################################
##

;
##################################################################
##################################################################
##



proc inputAmountCode

    printString blank2

    printString moneymsg5

    printString blank

    mov ah, 1

    int 21h

    mov inputAmountOption, al

    ret

inputAmountCode endp


withdraw proc

    call checkAccountCreated

    call getPinInput


    call clearScreen
```

```
printString withdrawmsg

printString blank2
printString moneymsg1
printString moneymsg2
printString moneymsg3
printString moneymsg4

call inputAmountCode

cmp inputAmountOption, '1'
je withdraw_1000

cmp inputAmountOption, '2'
je withdraw_2000

cmp inputAmountOption, '3'
je withdraw_5000

cmp inputAmountOption, '4'
je withdraw_10000
```

```
withdraw_1000:
    mov bx, totalAmount
    cmp bx, 1000
    jl lessMoney
    sub totalAmount, 1000
    printString moneymsg7
    printString blank2
    jmp mainLoop

withdraw_2000:
    mov bx, totalAmount
    cmp bx, 2000
    jl lessMoney
    sub totalAmount, 2000
    printString moneymsg7
    printString blank2
    jmp mainLoop

withdraw_5000:
    mov bx, totalAmount
```

```
    cmp bx, 5000
    jl lessMoney
    sub totalAmount, 5000
    printString moneymsg7
    printString blank2
    jmp mainLoop

withdraw_10000:
    mov bx, totalAmount
    cmp bx, 10000
    jl lessMoney
    sub totalAmount, 10000
    printString moneymsg7
    printString blank2
    jmp mainLoop

lessMoney:
    printString blank2
    printString moneymsg6
    printString mainmsg7
    printString blank2
```

```
        call etc


    ret

withdraw endp


;
##################################################################
##################################################################
##

;
##################################################################
##################################################################
##

;  DEPOSIT MONEY

;
##################################################################
##################################################################
##

;
##################################################################
##################################################################
##




deposit proc

    call checkAccountCreated
```

```
call getPinInput
call clearScreen

printString DEPOSITMSG;

printString blank2
printString moneymsg1
printString moneymsg2
printString moneymsg3
printString moneymsg4

call inputAmountCode

cmp inputAmountOption, '1'
je deposit_1000

cmp inputAmountOption, '2'
je deposit_2000

cmp inputAmountOption, '3'
je deposit_5000
```

```
cmp inputAmountOption, '4'
je deposit_10000

deposit_1000:
    add totalAmount, 1000
    printString moneymsg8
    printString blank2
    jmp mainLoop

deposit_2000:
    add totalAmount, 2000
    printString moneymsg8
    printString blank2
    jmp mainLoop

deposit_5000:
    add totalAmount, 5000
    printString moneymsg8
    printString blank2
    jmp mainLoop
```

```
    deposit_10000:
        add totalAmount, 10000
        printString moneymsg8
        printString blank2
        jmp mainLoop

    ret
deposit endp

;
##################################################################
##################################################################
##
;
##################################################################
##################################################################
##
; RESET ACCOUNT
;
##################################################################
##################################################################
##
;
##################################################################
```

```
##################################################################
##


reset proc
    call checkAccountCreated
    call getPinInput
    call clearScreen


    printString resetmsg;


    mov si, offset accountName
    mov cx, 30
    l1:
        mov [si],' '
        inc si
    loop l1


    mov cx, 30
    mov si, offset accountPIN
    l2:
        mov [si],' '
        inc si
```

```
    loop l2

    mov totalAmount, 0
    mov accountPINcount, 0

    printString blank2
    printString resetDone
    printString mainmsg7
    printString blank2
    call etc
    ret
reset endp

;
##################################################################
##

;
##################################################################
##

;  UPDATE

;
##################################################################
```

```
################################################################
##

;
################################################################
################################################################
##


macro modify_account str

    mov si, offset str


    modify_account_in:

        mov ah, 1

        int 21h

        cmp al, 13

        je mod_pin

        mov [si], al

        inc si

        jmp modify_account_in

endm


macro modify_pin str

    mov si, offset str

    mov accountPINcount, 0
```

```
    modify_pin_in:
        mov ah, 1
        int 21h
        cmp al, 13
        je modify_success
        inc accountPINcount
        mov [si], al
        inc si
        jmp modify_pin_in
endm

modify proc
    call checkAccountCreated
    call getPinInput
    call clearScreen

    printString modmsg

    ; Account name modify
    printString modAccMsg1
```

```
    printString accountName
    printString modAccMsg2
    printString blank
    modify_account accountName
    mod_pin:
        printString blank2
        printString modPinMsg1
        printString accountPIN
        printString modPinMsg2
        printString blank
        modify_pin accountPIN

    modify_success:
        call clearScreen
        printString create4
        printString modSuccess
        call etc
modify endp
```

;;;

```
;                                                    ;
;                ENTRY   POINT                       ;
;                                                    ;
;;;
```

```
call inputMenu

cmp inputCode, '0'
je exit

cmp inputCode, '1'
je create_account

cmp inputCode, '2'
je print_details

cmp inputCode, '3'
je withdraw

cmp inputCode, '4'
je deposit

cmp inputCode, '5'
je reset

cmp inputCode, '6'
```

```
        je modify


        jmp mainLoop
    exit:
        printString blank2
        printString blank2
        call displayBye
        printString blank2
        printString blank2


        mov ah,4ch
        int 21h
    main endp
end main
```